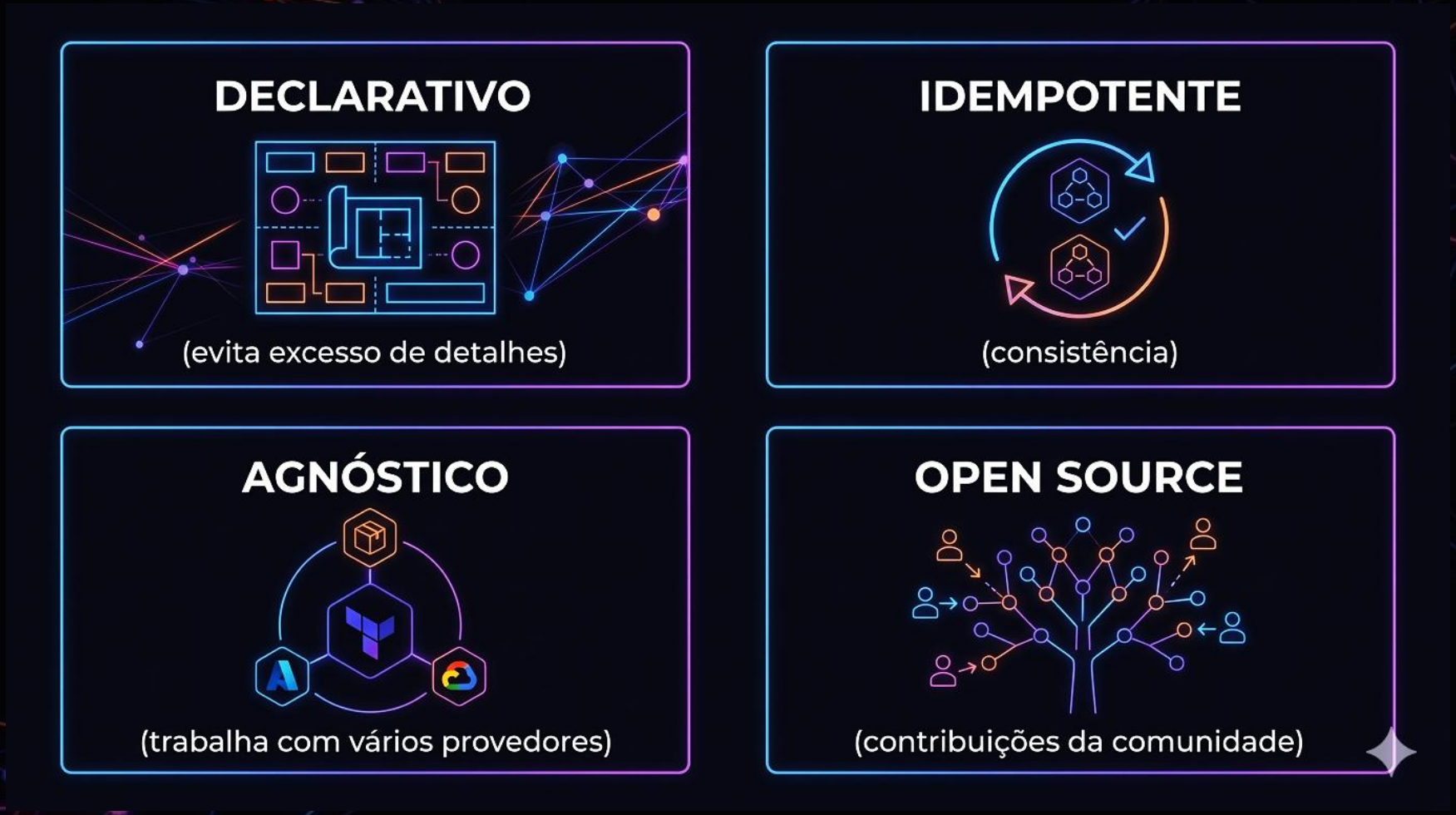
Introdução ao Terraform

- Infrastructure as Code (IaC) adotado em larga escala
- Adoção de Terraform (HashiCorp) cresce rapidamente



Introdução – “Getting Started”

Você pode instalar localmente e rodar com VS CODE / CMD
https://www.terraform.io/downloads

Comandos básicos:

```
C:\TEMP\terraform init
C:\TEMP\terraform validate   #seus scripts
C:\TEMP\terraform plan       #plano de execução dos scripts
C:\TEMP\terraform apply      #executa scripts
C:\TEMP\terraform destroy    #caso necessite fazer “rollback”
C:\TEMP\terraform show       #mostra resultado da implementação
```

Introdução – Scripts

Usa Linguagem HCL ou JSON
Permite escolher seu(s) provedor(es) de nuvem:

Para AZURE, use Azure RM
https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs

- Cria/altera recursos apenas informando novos valores
- Usa variáveis ao invés de valores fixos
- Permite criar scripts mais legíveis (script único vs múltiplos scripts)
- Implementação de recursos é feita em paralelo

Introdução – Sintaxe Básica

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~>3.0"
      ##any azurerm version, from 3.0.0
or above
    }}}
```

Scripts começam com a declaração de provedores e suas versões

Identifica parâmetros dos recursos a serem implementados (ex: https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/resource_group)

```
resource "azurerm_resource_group" "rg" {
  name     = “wagnerresourcegroup2024"
  location = "East Us"
}
```

Introdução – Como Executar Scripts

```
cd <path_to_scripts>
az login
rem to deploy to specific subscription run next command
az account set --subscription <subscriptionID>
terraform init
terraform validate
terraform plan  [-out]  <repositório/arquivoPlan.tfplan>
terraform apply [-auto-approve] <repositório/arquivoPlan.tfplan>
```

Controle de Estado

Após primeira execução dos scripts, novos arquivos serão criados

Arquivo TFSTATE identifica o estado de cada recurso após última execução

Antes da próxima execução, Terraform vai checar TFSTATE.



NUNCA PERCA OU ALTERE TFSTATE

Controle de Estado – Como Alterar ou Excluir Recursos

Arquivo TFSTATE é a chave para implementações bem sucedidas
Resultados esperados para próximas execuções de seus scripts:

- Se seus scripts não forem alterados – nada acontece (IDEMPOTENT)
- Se forem alterados valores de variáveis – recursos afetados serão alterados
- Se um recurso for excluído dos scripts – recurso será excluído
- Se o TFSTATE for perdido – TERRAFORM assume que os recursos não existem e tentará recriá-los (gerando mensagem de erro na implantação)



Uma Estratégia Melhor

Dividir seu código em múltiplos scripts costuma ser uma ótima opção:

- Melhora gerenciamento (ex: em caso de atualização do AzureRM)
- Melhora escalabilidade (ex: executar implementações em paralelo)
- Melhora reusabilidade (ex: aplicação em ambientes Dev/Tst/Prd)

Uma Estratégia Melhor – Tipos de Scripts

Na prática, é uma boa ideia dividir seu código entre vários scripts
Seu projeto pode incluir (por exemplo):

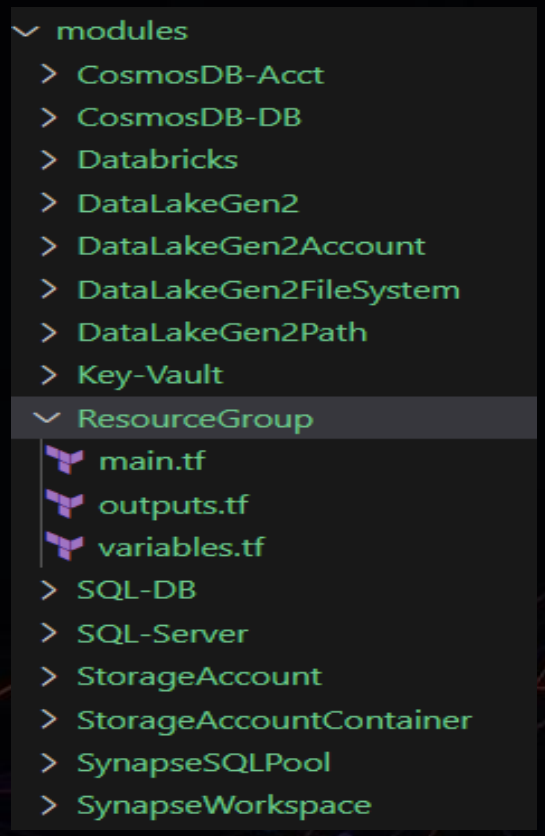
- [main].tf : script raiz; normalmente provedores e recursos a serem criados.
- variables.tf : onde se define as variáveis, tipos de dados e comentários
- locals.tf : cria expressões baseadas em variáveis (exemplo: criação de “tags”)
- output.tf : coleta valores de atributos durante a criação dos recursos (ex: IDs).
- custom files : arquivos genéricos para customização do seu código
- MÓDULOS : bibliotecas que encapsulam sintaxe de criação de cada recurso

Uma Estratégia Melhor – Porque Usar Módulos?

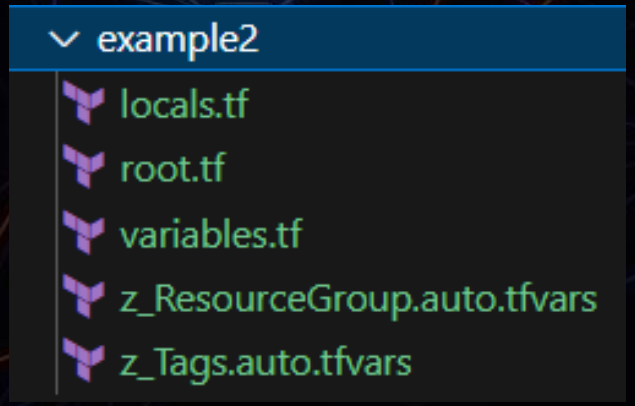
- Eles isolam a sintaxe correta de criação de cada recursos
- Essa biblioteca deve ser compartilhada com todo time de DEV
- Define um padrão corporativo
- Melhor gerenciamento de upgrades e mudanças de sintaxe dos recursos

Uma Estratégia Melhor – Organizando Scripts Para Usar Módulos

Crie um diretório de MÓDULOS



Crie diretórios de PROJETOS



Uma Estratégia Melhor – Conteúdo dos Scripts: Módulo

modules > ResourceGroup > main.tf

```
#reference https://registry.terraform.io/providers/h
resource "azurerm_resource_group" "ResourceGroup" {
    name                = var.rg_name
    location            = var.location
    tags                = var.tags
}
```

modules > ResourceGroup > outputs.tf

```
output "newResourceGroupName" {
  value = azurerm_resource_group.ResourceGroup.name
}

output "id" {
  value = azurerm_resource_group.ResourceGroup.id
}

#output "tenant_id" {
#  value = azurerm_resource_group.ResourceGroup.tenant_id
#}
```

modules > ResourceGroup > variables.tf

```
variable "rg_name" {
    type  = string
    description = "resource group name"
}

variable "location" {
    type  = string
    description = "code related to Azure Region"
}


variable "tags" {
    type  = map(string)
    description = "tags related to project"
}
```

Uma Estratégia Melhor –
Conteúdo dos Scripts: Arquivos do Projeto

example2 > root.tf

```
1   # Terraform and Azure Provider configuration
2   terraform {
3     required_providers {
4       azurerm = {
5         source = "hashicorp/azurerm"
6         version = "~>3.0"
7         ##any azurerm version, from 3.0.0 or above
8       }
9     }
10  }
11  provider "azurerm" {
12    features {}
13  }
14
15  module "ResourceGroup1" {
16    source      = "../modules/ResourceGroup"
17    rg_name     = local.rgname1
18    location    = var.location1
19    tags        = local.tag
```

example2 > variables.tf

```
33  # ##########################################################
34  # variables related to RESOURCE GROUP
35  # ##########################################################
36  variable "resourceIdentifier" {
37      type  = string
38      description = "preffix to resource group name"
39  }
40
41  variable "location1" {
42      type  = string
43      description = "code related to Azure Region"
44  }
45
46  variable "shortlocation1" {
47      type  = string
48      description = "short code related to Azure Region"
49  }
```

Uma Estratégia Melhor – Conteúdo dos Scripts: TFVARS & Locals

example2 > z_Tags.auto.tfvars

```
company                 = "WWI"
project                 = "platdt"
costcenter              = "xyz"
environment             = "dev"
shortprovider           = "az"
resource_id             = "10"

```

example2 > z_ResourceGroup.auto.tfvars

```

resourcegroup_identifier= "rg"
location                = "southcentralus"
shortlocation           = "ussc"
```

example2 > locals.tf

```

locals {

  rgname1 = format("%s-%s-%s-%s-01"
      , var.shortprovider, var.shortlocation1
      , var.environment, var.resourceIdentifier)

  tag = {
    company       = var.company
    project       = "${var.company}-${var.project}"
    costcenter    = var.costcenter
    environment   = var.environment
  }
}
```

Múltiplos Recursos

- Atenção às convenções de nomenclatura

  https://bit.ly/azurenames

- Esteja atento a nomes únicos
- Considere dependências entre recursos
- Lembre-se que seu script pode combinar o uso de múltiplos provedores



Múltiplos Recursos – Sobre os Nomes Únicos

- No Azure, os recursos precisam ter nomes únicos dentro do seu escopo
- Alguns recursos tem escopo global, portanto é necessário atenção
- Uma alternativa é adicionar um valor aleatório para gerar um nome único (locals.tf)

locals.tf

```
1   #Random ID for unique naming
2   resource "random_integer" "rand" {
3     min = 000001
4     max = 999999
5   }
6
7   locals {
8     randomSuffix = "${format("%06s", random_integer.rand.result)}"
9     rgname = "${var.rgPreffix}${var.rgName}-${local.randomSuffix}"
10  }
```

Múltiplos Recursos – Dependências Entre Recursos

Muitos recursos dependem de um recurso “pai”

Nesses casos, esteja atento a 2 pontos principais:

O recurso “filho” deve ser implementado DEPOIS do “pai”.

Garanta que o “pai” forneça as informações necessárias para definição do recurso “filho” (outputs.tf)

modules > SynapseWorkspace > outputs.tf

```
1  output "name" {
2    description = "The name of the resoruce created."
3    value       = azurerm_synapse_workspace.synapsewokspace.name
4  }
5
6  output "id" {
7    description = "The id of the resource created."
8    value       = azurerm_synapse_workspace.synapsewokspace.id
9  }
```

example3 > root.tf

```
230  module "synapsesqlpool" {
231    source                = "../modules/SynapseSQLPool"
232    synsqlpool_name       = var.synsqlpool_name
233    synsqlpool_wrkspcid   = module.synapseworkspace.id
234    synsqlpool_sku        = var.synsqlpool_sku
235    synsqlpool_mode       = var.synsqlpool_mode
236
237    depends_on = [module.synapseworkspace]
238  }
```

Múltiplos Recursos – Mensagens Exibidas

Terraform apresentação mensagens de status durante a implementação (CMD)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [3m50s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [4m0s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [4m10s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [4m20s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [4m30s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [4m40s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Creation complete after 4m49s [id=/subscriptions/56508de2-630e-4d64-a40f-45bfa674520b/resourceG
roups/az-ussc-dev-rg-563645/providers/Microsoft.Synapse/workspaces/az-ussc-dev-syn-563645]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Creating...
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [10s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [20s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [30s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [40s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [50s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m0s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m10s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m20s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m30s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m40s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m50s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m0s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m10s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m20s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m30s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m40s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m50s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Creation complete after 2m54s [id=/subscriptions/56508de2-630e-4d64-a40f-45bfa674520b/resourceGroup
s/az-ussc-dev-rg-563645/providers/Microsoft.Synapse/workspaces/az-ussc-dev-syn-563645/sqlPools/whiterabbit]

Apply complete! Resources: 17 added, 0 changed, 0 destroyed.
PS C:\temp\Terraform101\example3>
```

REFERÊNCIAS

GitHub – All Files Presented in this Workshop:
https://github.com/wcrivelini/articles/tree/main/Azure_Terraform

Database Deployment with Terraform - The Basics:
https://www.sqlservercentral.com/articles/database-deployment-with-terraform-the-basics

Database Deployment with Terraform – Modules:
https://www.sqlservercentral.com/articles/database-deployment-with-terraform-modules